



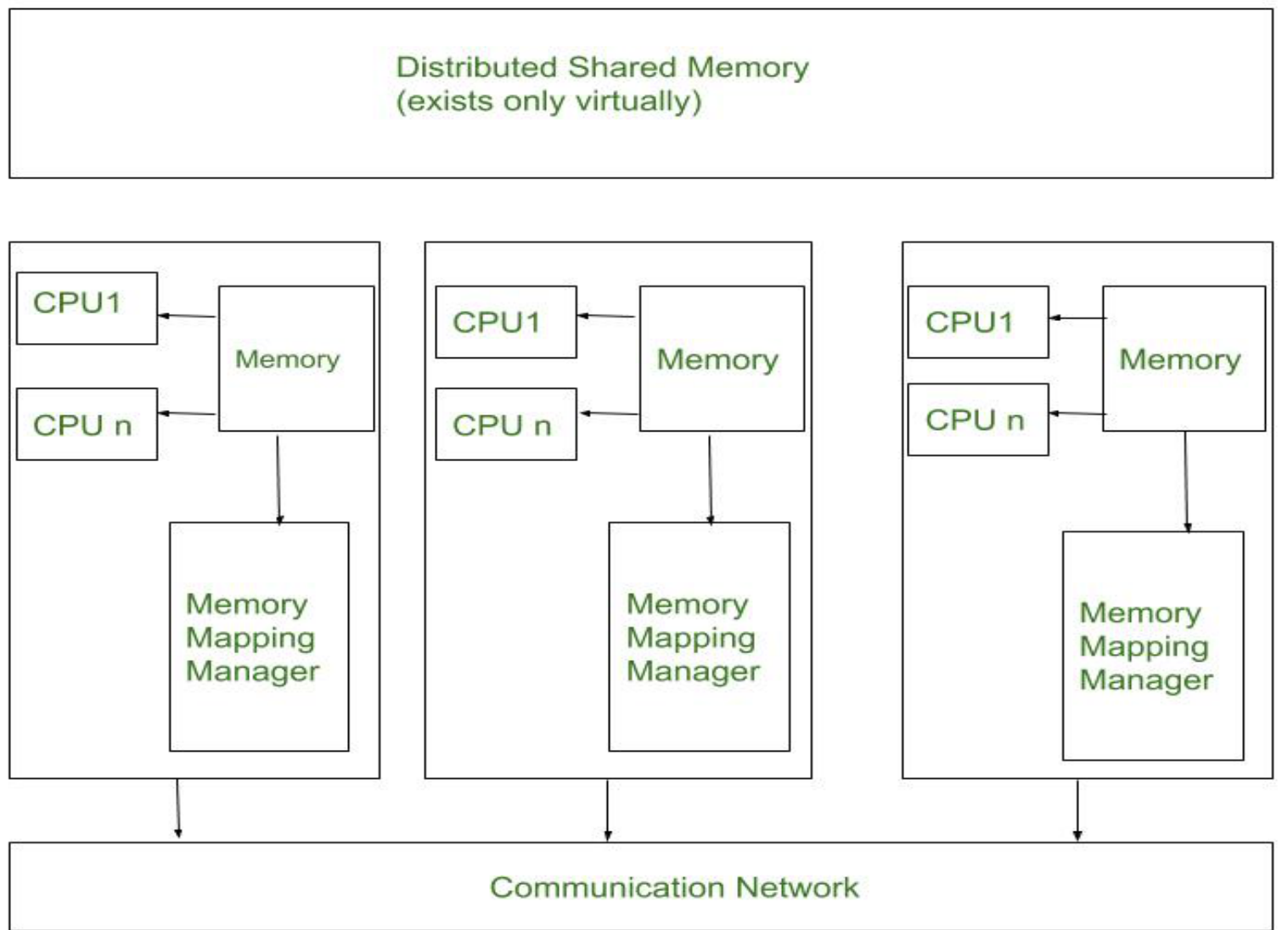
# Consistency and Replication

## Distributed Shared Memory

# Distributed Shared Memory

- For Interprocess communication two basic paradigms:
  - Message passing (RPC)
  - Shared-memory [Two primitives: Send(recipient, data) & Receive (data)]
  - The sending process generates the data to be shared and sends it to the recipient with which it wants to communicate.
- The term DSM refers to the shared-memory paradigm applied to loosely coupled distributed memory system.
- DSM is an abstraction used for sharing data between computers that do not share physical memory.
- Processes access DSM by reads and updates to what appears to be ordinary memory within their address space.
- However, an underlying runtime system ensures transparently that processes executing at different computers observe the updates made by one another.

# Distributed Shared Memory



# DSM Architecture

- As shown in the Figure, DSM provides a virtual address space shared among processes on loosely coupled processors.
- i.e., DSM is basically an abstraction that integrates the local memory of different machines in a network environment into a single logical entity shared by cooperating processes executing on multiple sites.
- The shared memory itself exists only virtually.
- Application programs can use it in the same way as a traditional virtual memory, except that processes can run on different machines in parallel.
- Due to the virtual existence of the shared memory, DSM is sometimes also referred to as Distributed Shared Virtual Memory (DSVM).

# DSM Architecture

- Each node of the system consists of **one or more CPUs** and a **memory unit**. The nodes are connected by a **high-speed communication network**.
- A simple **message-passing** system allows processes on different nodes to **exchange messages** with each other.
- The DSM abstraction presents a large shared-memory space to the processors of all nodes (DSM exists only virtually).
- **A software memory-mapping manager** routine in each node maps the local memory onto the shared virtual memory. To facilitate the mapping operation, the shared-memory space is partitioned into blocks.

# DSM Architecture

- Data caching is a well-known solution to address memory access latency.
- The idea of data caching is used in DSM systems to reduce network latency. That is, the main memory of individual nodes is used to cache pieces of the shared-memory space.
- The memory-mapping manager of each node views its local memory as a big cache of the shared-memory space for its associated processors. The basic unit of caching is a memory block.
- When a process on a node accesses some data from a memory block of the shared memory space, the local memory-mapping manager takes charge of its request.
- If the memory block containing the accessed data is resident in the local memory, the request is satisfied by supplying the accessed data from the local memory.
- Otherwise, a network block fault is generated and the control is passed to the operating system.

# DSM Architecture

- The operating system then sends a message to the node on which the desired memory block is located to get the block.
- The missing block is migrated from the remote node to the client process's node and the operating system maps it into the application's address space.
- The faulting instruction is then restarted and can now complete. Therefore, the scenario is that data blocks keep migrating from one node to another on demand but no communication is visible to the user processes.
- That is, to the user processes, the system looks like a tightly coupled shared-memory multiprocessors system in which multiple processes freely read and write the shared-memory at will, Copies of data cached in local memory eliminate network traffic for a memory access on cache hit, that is, access to an address whose data is stored in the cache. Therefore, network traffic is significantly reduced if applications show a high degree of locality of data accesses.
- Variations of this general approach are used in different implementations depending on whether the DSM system allows replication and/or migration of shared-memory data blocks.

# Issues of Distributed Shared Memory

- Granularity
- Structure of Shared-Memory Space
- Memory Coherence and Access Synchronization
- Data Location and Access
- Replacement Strategy
- Thrashing
- Heterogeneity

# Issues of Distributed Shared Memory

- **Granularity:** Granularity refers to the block size of a DSM system, that is, to the unit of sharing and the unit of data transfer across the network when a network block fault occurs.
- Possible units are a few words, a page, or a few pages. Selecting proper block size is an important part of the design of a DSM system because block size is usually a measure of the granularity of parallelism explored and the amount of network traffic generated by network block faults.
- **Structure of Shared-Memory Space:** Structure refers to the layout of the shared data in memory.
- The structure of the shared-memory space of a DSM system is normally dependent on the type of applications that the DSM system is intended to support.

# Issues of Distributed Shared Memory

- **Memory Coherence and Access Synchronization:** In a DSM system that allows replication of shared data items, copies of shared data items may simultaneously be available in the main memories of a number of nodes.
- In this case, the main problem is to solve the memory coherence problem that deals with the consistency of a piece of shared data lying in the main memories of two or more nodes.
- This problem is similar to that which arises with conventional caches in particular with multicache schemes for shared-memory multiprocessors
- Since different memory coherence protocols make different assumptions and trade-offs, the choice is usually dependent on the pattern of memory access.
- The terms coherence and consistency are used interchangeably in the literature.
- In a DSM system, concurrent accesses to shared data may be generated. Therefore, a memory coherence protocol alone is not sufficient to maintain the consistency of shared data.
- In addition, synchronization primitives, such as semaphores, event count, and lock, are needed to synchronize concurrent accesses to shared data.

# Issues of Distributed Shared Memory

- **Data Location and Access:** The mechanism of locating, retrieving and transferring of requested data in DSM by user process needs to be properly designed to respond to block faults effectively by availing consistent data image.
- **Replacement Strategy:** If the local memory of a node is full, a data block of the local memory must be replaced by the new data block. Therefore, a cache replacement strategy is also necessary in the design of a DSM system.
- Proper replacement strategy is required in the situation like whole local cache is fully occupied and the process has to respond to any data transfer request generated by any node on DSMS

# Issues of Distributed Shared Memory

- **Thrashing:** This is the problem that emerges in DSMS when two processes from two different nodes request the same data block. In this situation the data block is moved back and forth among these competing processes quickly, the quickness is as fast as it seems no data transfer was carried out. A DSM system must use a policy to avoid this situation.
- **Heterogeneity:** If the nodes and their underlying architecture used to design DSMS are homogeneous then no heterogeneity issues can emerge. However, if the underlying environment is different then the heterogeneity takes place. The DSMS architecture design should facilitate positive and productive data communication when DSMS is heterogeneous in nature.



# Consistency and Replication

# Motivation

- Make copies of services on multiple sites, improve ...
  - **Reliability** (by redundancy)
    - If primary File Server crashes, standby File Server still works
  - **Performance**
    - Increase processing power
    - Reduce communication delays
  - **Scalability**
    - Prevent overloading a single server (size scalability)
    - Avoid communication latencies (geographic scale)
- However, updates are more complex
  - When, who, where and how to propagate the updates?

# Why Replicate Data?

- Enhance reliability.
- Improve performance.
- But: if there are many replicas of the same thing, how do we keep all of them up-to-date? How do we keep the replicas *consistent*?
- Consistency can be achieved in a number of ways.
- There are number of *consistency models*, as well as *protocols* for implementing the models.

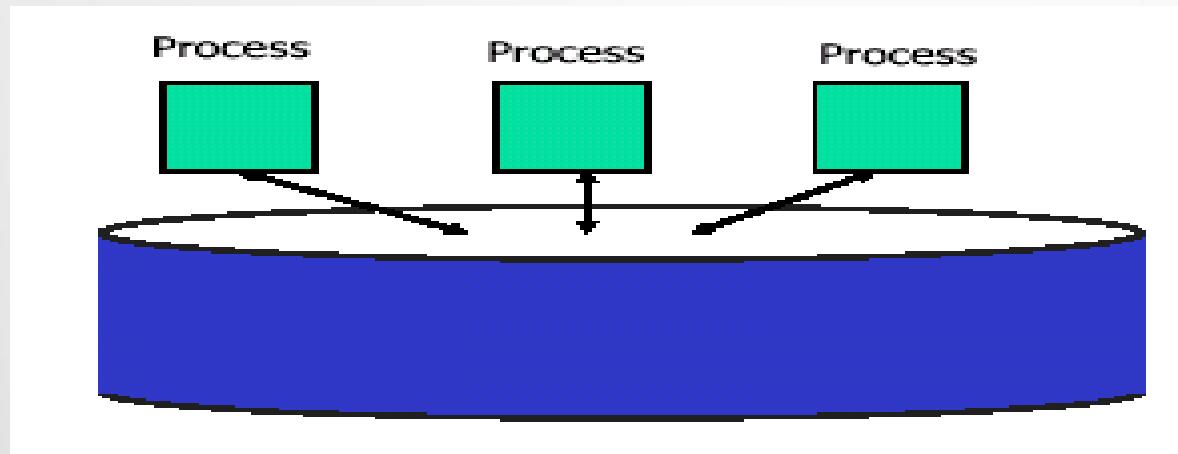
# Replication (Uses)

- Replicas allows remote sites to **continue working** in the event of **local failures**.
- It is also possible to **protect against data corruption**.
- Replicas allow **data to reside close** to where it is used.
- This directly supports the distributed systems goal of enhanced ***scalability***.
- Even a large number of replicated “local” systems can improve performance: clusters.
- The issue: It is **not easy** to keep all those replicas ***consistent***.



# Consistency models

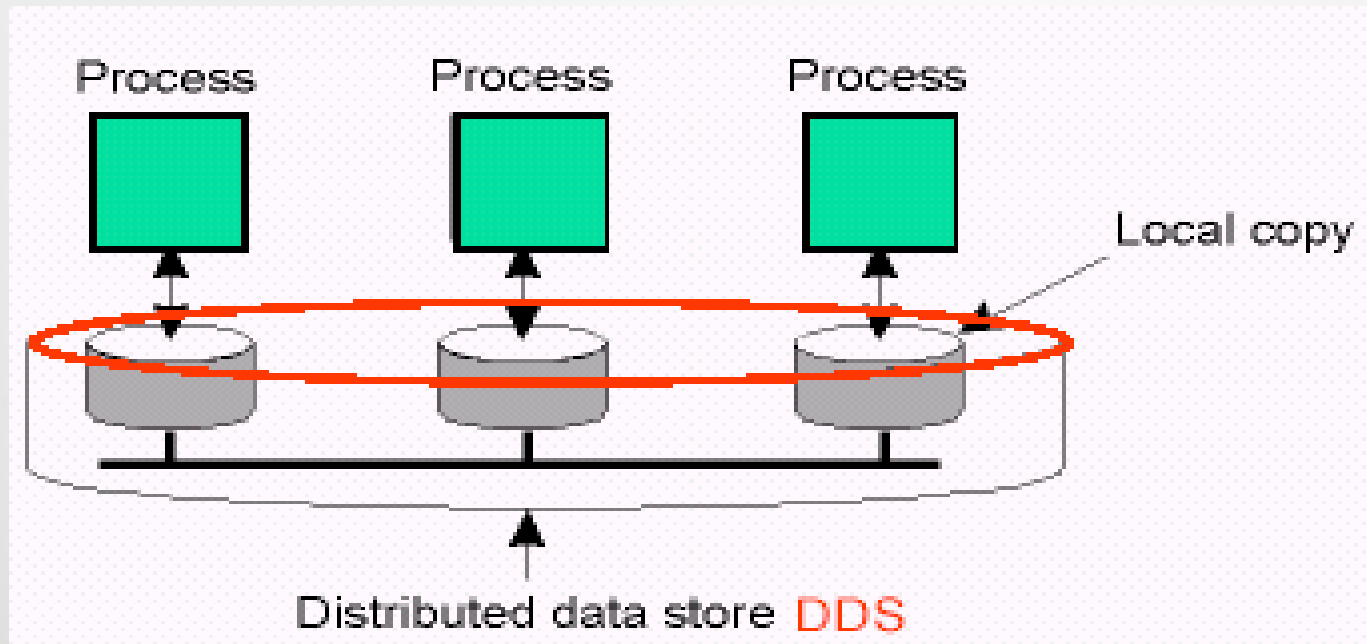
# Distributed Data Store



Clients point of view:

The data store is capable of storing huge amount of data

# Distributed Data Store



## Data Store's point of view:

General organization of a logical data store, physically distributed and replicated across multiple tasks.

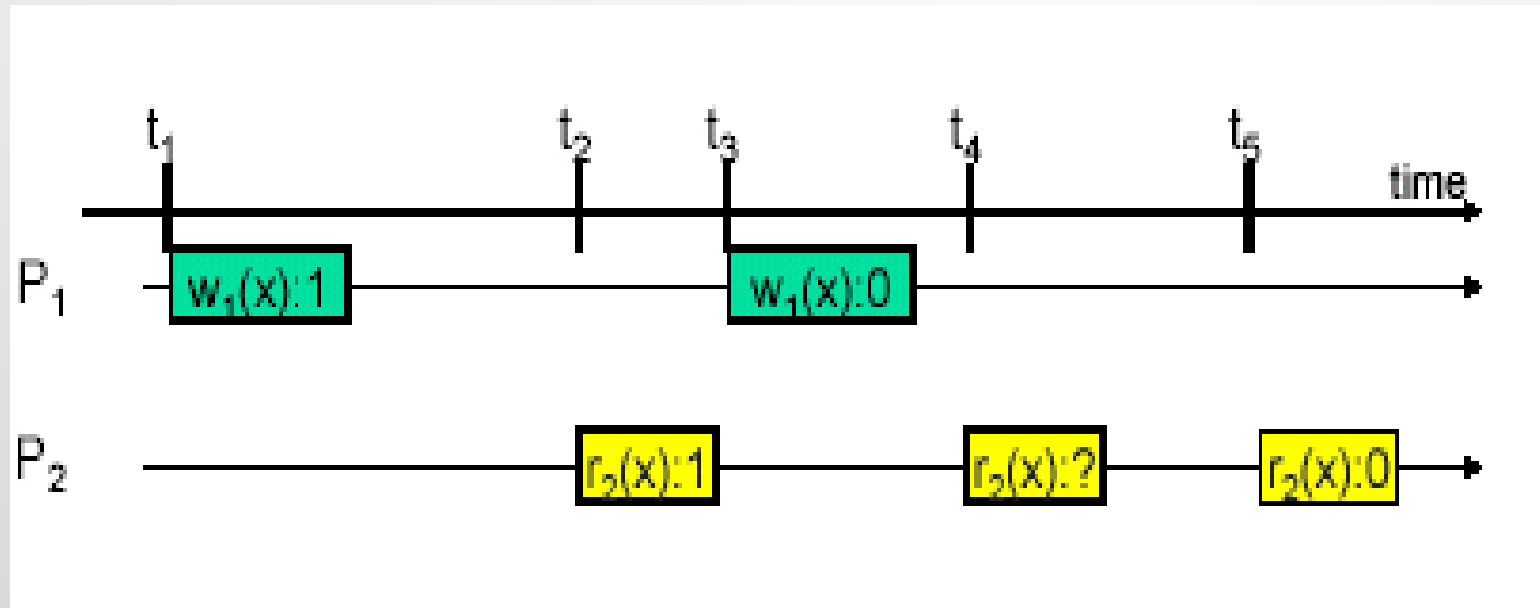
# Distributed Data Store

- A data-store can be read from or written to by any process in a distributed system.
- A local copy of the data-store (replica) can support “fast reads”.
- However, a **write** to a local replica needs to **be propagated to *all* remote replicas**.
- Various consistency models help to understand the various mechanisms used to achieve and enable this.

# Operations on a Data Store

- **Read:  $r_i(x)b$**  client  $i$  or process  $P_i$  performs a read for data item  $x$  and it returns value  $b$
- **Write:  $w_i(x)a$**  client  $i$  or process  $P_i$  performs a write on data  $x$  setting it to the new value  $a$
- **Operations not instantaneous: they depend on**
  - Time of issue (when request is sent by client)
  - Time of execution (when request is executed at a replica)
  - Time of completion (when reply is received by client)

# Example



Time moves from left to right in all diagrams.

**Main issue:** To keep replicas consistent, we generally need to ensure that all **conflicting** operations are done in the same order everywhere.

**Conflicting operations:** From the world of transactions:

- o **Read-write conflict:** a read operation and a write operation act concurrently
- o **Write-write conflict:** two concurrent write operations

Guaranteeing global ordering on conflicting operations may be a costly operation, downgrading scalability

**Solution:** weaken consistency requirements so that hopefully global synchronization can be avoided

# Consistency Models

- Defines which interleaving of operations is **valid** (admissible)
- Different levels of consistency
  - **strong** (strict, tight)
  - **weak** (loose)
- **Consistency model:**
  - Concerned with the consistency of a data store
  - Specifies characteristics of valid ordering of operations
- A data store that implements a **particular consistency model** will provide a **total ordering** of operations that is **valid** according to this model

# Consistency Models

- A “consistency model” is a CONTRACT between a data-store and its processes.
- If the processes agree to the rules, the data-store will perform properly and as advertised.
- Data-centric models
  - Described consistency experienced by all clients
  - Clients P1, P2, P3, ... see **same** kind of **orderings**
- Client centric models:
  - Described consistency only seen by clients who request it
  - Clients P1, P2, P3 may see **different** kinds of orderings

# Data-Centric Consistency Models

- Strong ordering:
  - Strict consistency
  - Sequential consistency
  - Linear consistency
  - Causal consistency
  - FIFO consistency
- Weak ordering:
  - Weak consistency
  - Release consistency
  - Entry consistency

# Strict Consistency

- Definition: A DDS (distributed data store) is strict consistent if any read on a data item  $x$  returns the value corresponding to the result of the most recent write on  $x$ , regardless of the location of the processes doing read or write
- Analysis:
  - In a single processor system strict consistency is for nothing, that's exact the behavior of local shard memory with atomic reads/writes
  - However, it's hard to establish a global time to determine what's the most recent write
  - Due to message transfer delays this model is not achievable

# Strict Consistency Diagrams

- Behavior of two processes, operating on the same data item:

a) A strictly consistent data-store.      b) A data-store that is not strictly consistent.

|       |       |       |
|-------|-------|-------|
| P1:   | W(x)a |       |
| <hr/> |       |       |
| P2:   |       | R(x)a |

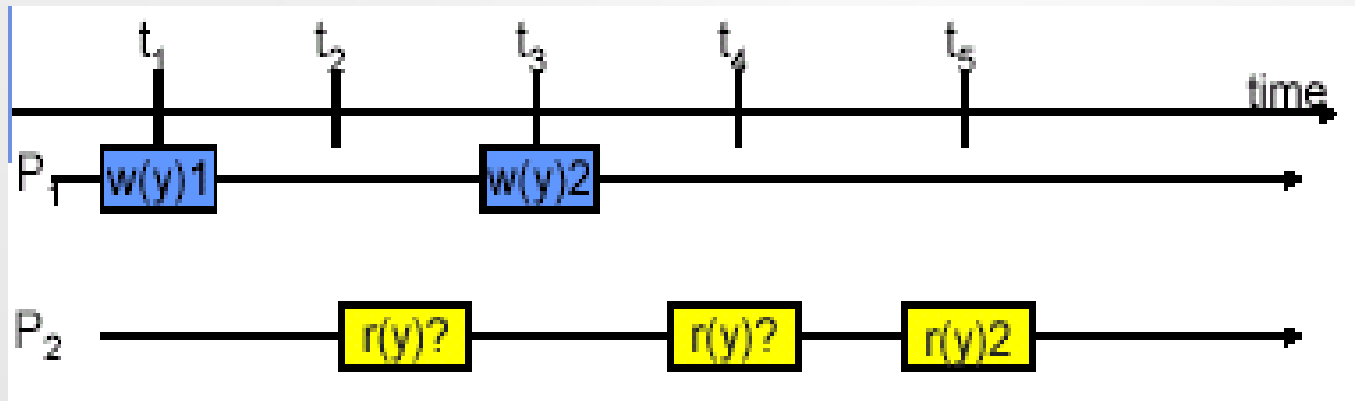
(a)

|       |       |                  |
|-------|-------|------------------|
| P1:   | W(x)a |                  |
| <hr/> |       |                  |
| P2:   |       | R(x)NIL    R(x)a |

(b)

- The update must be propagated to all other replicated copies.
- If P2 reads and finds this value to be NIL (update did not propagate due to propagation delay) and then at some time later gets the value and reads it to be a, as in fig(b).
- The results read by P2 are not consistent with strict consistency models.

# Strict Consistency Problems



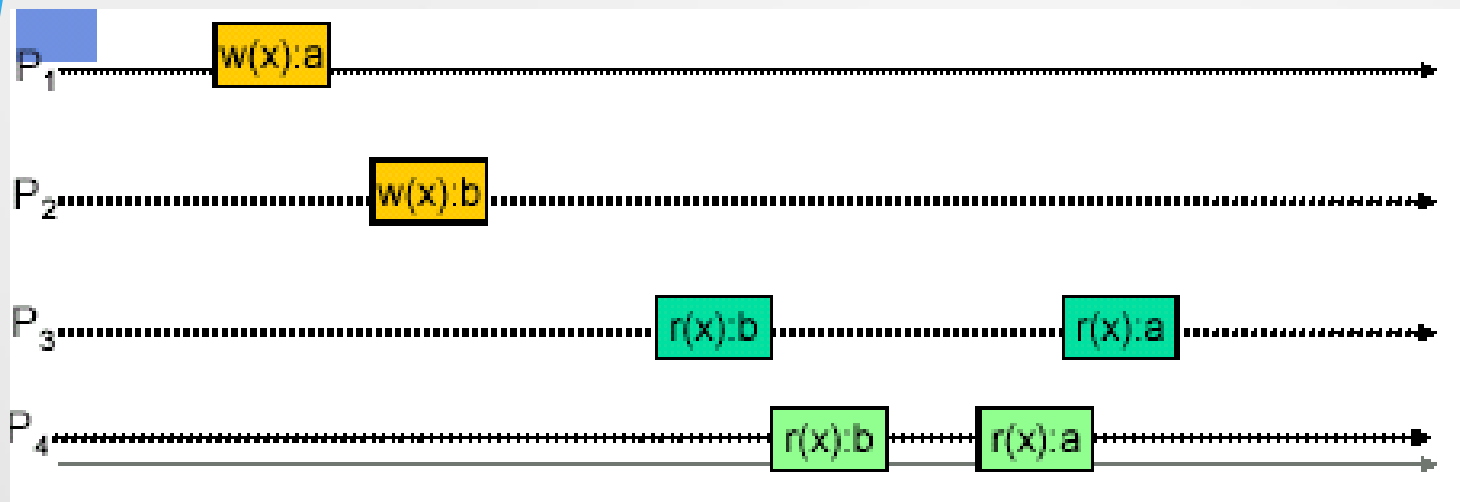
Assumption:  $y = 0$  is stored on node 2,  $P_1$  and  $P_2$  are processes on node 1 and 2,

Due to message delays,  $r(y)$  at  $t = t_2$  may result in 0 or 1 and at  $t = t_4$  may result in 0, 1 or 2.

|       |              |              |              |              |
|-------|--------------|--------------|--------------|--------------|
| $P_1$ | $W_1(x) 100$ |              |              |              |
| $P_2$ |              | $W_2(y) 200$ |              |              |
| $P_3$ |              |              | $R_3(y) 200$ | $R_3(x) 100$ |
| $P_4$ |              |              | $R_4(x) 100$ | $R_4(y) 200$ |

# Sequential Consistency

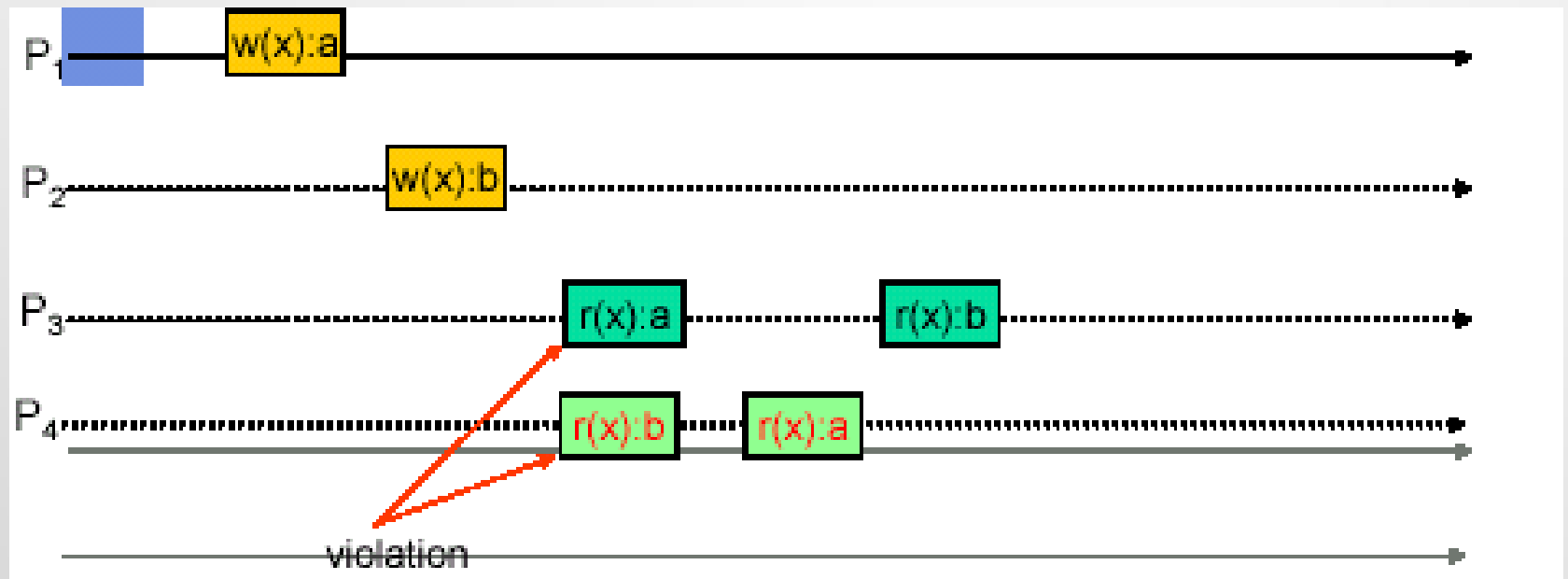
- Definition: *The result of any execution is the same as if the (read and write) operations by all processes on the data store were executed in the some sequential order and the operations of each individual process appear in this sequence in the order specified by its program.*
- *Informally, sequential consistency implies that operations appear to take place in some total order, and that order is consistent with the order of operations on each individual process.*
- Sequential consistency is proposed by Lamport and is weaker than strict consistency



## Example

- P<sub>1</sub> performs a write  $W(x)a$  on its local copy of the data store and changes to  $a$ .
- Later, P<sub>2</sub> performs a write operation and changes this value of  $x$  from  $a$  to  $b$ .
- both P<sub>3</sub> and P<sub>4</sub> may read this value to be  $b$  before reading it as  $a$  (because the update from P<sub>1</sub> did not get propagated, may be because of crash or some link breakage).
- This is a valid sequential consistency model as the updates are seen by all in the same order.
- Here write operation of process P<sub>2</sub>,  $W(x)b$ , appears to have been executed before  $W(x)a$  in P<sub>1</sub>.
- Each task sees all writes in the same order, even though not strict consistent.

# Non-Sequential Consistency



- The order in which processes P<sub>3</sub> and P<sub>4</sub> read the value is not the same.
- P<sub>3</sub> reads the value of x as b (`R(x)b`) and P<sub>4</sub> reads the value of x as a (`R(x)a`)); thus it violates the sequential consistency model.

# Linear Consistency

- Definition: A DDS is said to be linear consistent (linearizable) when each operation is time-stamped and the following holds: The result of each execution is the same as if the read and write operations by all processes on the DDS were executed in some sequential order and the operations of each individual process appear in this sequence in the order specified by its program. In addition, if  $TS_{OP1}(x) < TS_{OP2}(y)$ , then operation  $OP1(x)$  should precede  $OP2(y)$  in this sequence
- Fig. (a) depicting consistent sequential model is not linearizable consistent as  $P3$  and  $P4$  do not see the event of  $W(x)a$  at  $P1$  first, though timestamp of  $W(x)a$  is less than the timestamp of  $W(x)b$  happening at  $P2$ .

# Linear Consistency

- All servers execute all ops in some identical sequential order
- Global ordering preserves each client's own local ordering
- Global ordering preserves real-time guarantee
  - All ops receive global time-stamp using a sync'd clock
  - If  $tsop1(x) < tsop2(y)$ ,  $OP1(x)$  precedes  $OP2(y)$  in sequence
- Once write completes, all later reads should return value of that write or value of later write.
- Once read returns particular value, all later reads should return that value or value of later write.

# Assumption

- Each operation is assumed to receive a **time stamp** using a globally available clock, but with **only finite precision**, e.g. some loosely coupled synchronized local clocks.
- **Linear consistency is stricter than sequential one, i.e. a linear consistent DDS is also sequentially consistent.**
- With linear consistency no longer each valid interleaving of reads and writes is allowed, the ordering has also obey the order implied by the time-stamps of these operations.

# Causal Consistency

- Dilutes the constraints and eliminates the event ordering of the events which are concurrent and need not to be seen by all processes in any order
- Definition: Writes that are potentially causally related must be seen by all processes in the same order. Concurrent writes may be seen in a different order by different processes.
  - If event B is caused or influenced by an earlier event A, then causal consistency requires that every other process see event A, then event B.

# Causal Consistency

- Definition: write2 is potentially dependent on write1, when there is a read between these 2 writes which may have influenced write2
- Corollary: If write2 is potential dependent on write1  $\rightarrow$  the only correct sequence is: write1  $\rightarrow$  write2.

# Causal Consistency: Example

|     |       |       |       |       |
|-----|-------|-------|-------|-------|
| P1: | W(x)a |       | W(x)c |       |
| P2: |       | R(x)a | W(x)b |       |
| P3: |       | R(x)a |       | R(x)c |
| P4: |       | R(x)a |       | R(x)b |

- This sequence is allowed with a causally-consistent store, but not with sequentially or strictly consistent store.
- Note: it is assumed that  $W_2(x)b$  and  $W_1(x)c$  are concurrent.

# Causal Consistency: Example

|     |       |       |       |
|-----|-------|-------|-------|
| P1: | W(x)a |       |       |
| P2: |       | R(x)a | W(x)b |
| P3: |       |       | R(x)b |
| P4: |       |       | R(x)a |

(a)

|     |       |       |       |
|-----|-------|-------|-------|
| P1: | W(x)a |       |       |
| P2: |       | W(x)b |       |
| P3: |       |       | R(x)b |
| P4: |       |       | R(x)a |

(b)

- a) **Violation of causal-consistency** – P2's write is related to P1's write due to the read on 'x' giving 'a' (all processes must see them in the same order).
- b) **A causally-consistent data-store**: the read has been removed, so the two writes are now *concurrent*. The reads by P3 and P4 are now OK.

# Implementation

- Implementing causal consistency requires keeping track of which processes have seen which writes.
- Construction and maintenance of a **dependency graph**, expressing which operations are causally related (using vector time stamps)

# FIFO or PRAM Consistency

- Removes the constraint of having any relationship amongst the events happening on different processes.
- It just ensures that the global ordering is preserved for the **events happening in a single process**. It is also called “PRAM” (pipelined RAM)
- Definition: Writes by a single process are seen by all other processes in the order in which they were issued; however, writes from different processes may be seen in different orders by different processes.
  - Corollary: Writes on different processors are concurrent
- Implementation: Tag each write-operation of every process with: (PID, sequence number)

# FIFO Consistency Example

P1: W(x)a

P2: R(x)a W(x)b W(x)c

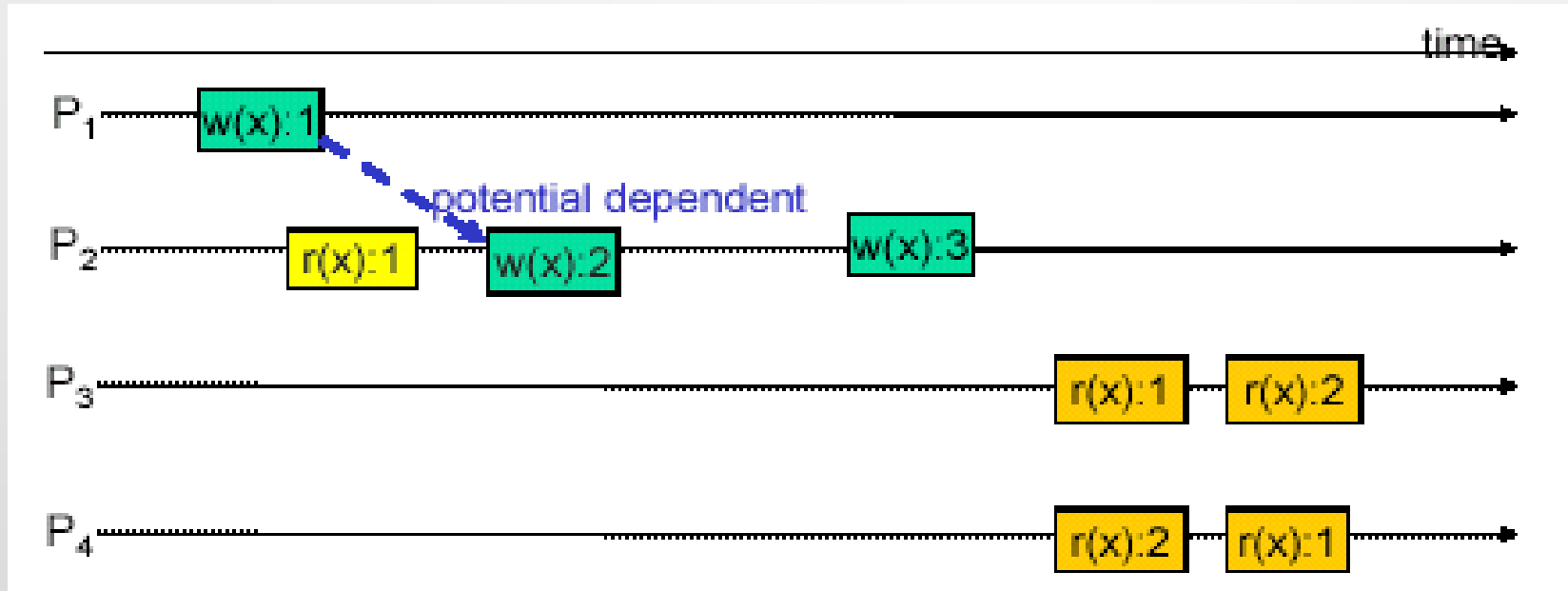
P3: R(x)b R(x)a R(x)c

P4: R(x)a R(x)b R(x)c

A valid sequence of FIFO consistency events.

- Note that none of the consistency models studied so far would allow this sequence of events.
- The attractive characteristic of FIFO is that it is easy to implement.
- There are no guarantees about the order in which different processes see writes – except that two or more writes from a single process must be seen in order.

# Example



Both writes are seen on processes  $P_3$  and  $P_4$  in a different order, they still obey FIFO-consistency, but not causal consistency because write 2 is dependent on write 1.

# Data-Centric Consistency Models

- Strong ordering:
  - Strict consistency
  - Sequential consistency
  - Linear consistency
  - Causal consistency
  - FIFO consistency
- Weak ordering:
  - Weak consistency
  - Release consistency
  - Entry consistency

# Weak Consistency

- Not all applications need to see all writes, let alone seeing them in the same order.
- This leads to “Weak Consistency” (which is primarily designed to work with *distributed critical regions*).
- This model introduces the notion of a “synchronization variable”, which is used to update all copies of the data-store.
- The process acquiring this synchronization variable is allowed to access the critical region and when it leaves the variable should be released

# Weak Consistency

- Definition: DDS implements weak consistency, if the following hold:
  - Accesses to synchronization variables “S” which is associated with a data store are sequentially consistent.
  - Operations on the synchronization variable are not allowed until all previous writes are performed everywhere.
  - No read or write operation can be performed on a data store until all previous synchronization operations to synchronization variables S are performed.

# Interpretation

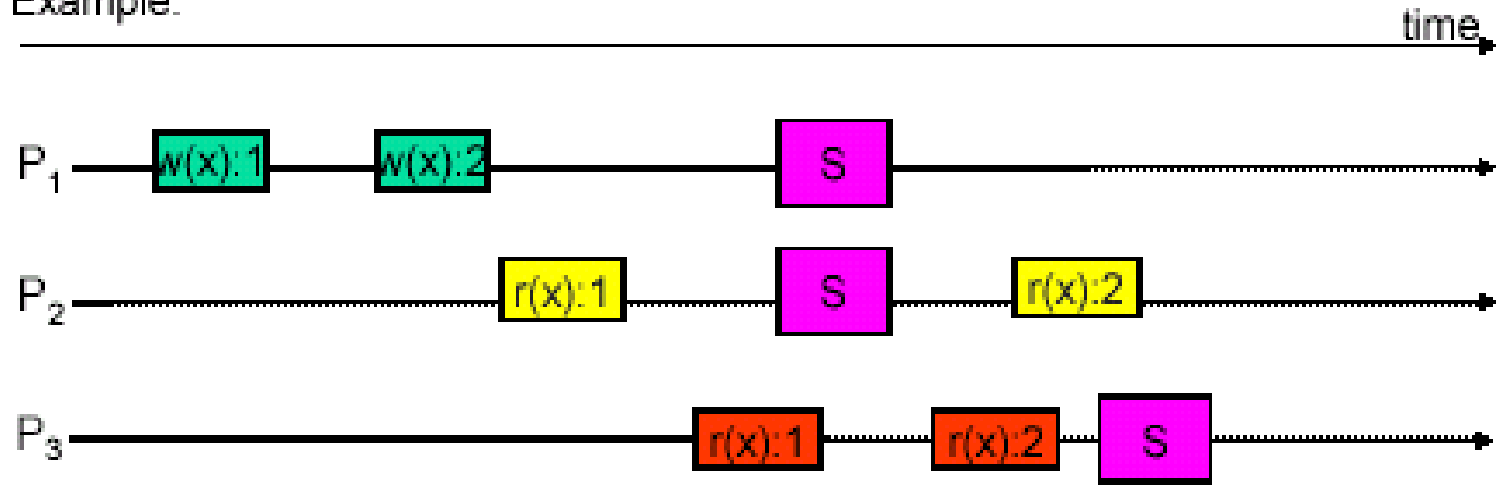
- A synchronization variable  $S$  knows just one operation: `synchronize(S)` responsible for all local replicas of the data store
- Whenever a process calls `synchronize(S)`, its local updates will be updated on all replicas of the DDS and all updates of the other processes will be updated to its local replica of the DDS
- All tasks see all accesses to synchronization-variables in the same order
- No data access allowed until all previous accesses to synchronization variables have been done
  - By doing a synch before reading shared data, a task can be sure of getting the “up to date value”
  - Unlike previous consistency models “weak consistency” forces the programmer to collect critical operations all together

# Weak Consistency

- By doing a sync.,
  - a process can *force* the just written value out to all the other replicas.
  - a process can be *sure* it's getting the most recently written value before it reads.
- In essence, the weak consistency models enforce consistency on a *group of operations*, as opposed to individual reads and writes (as is the case with strict, sequential, causal and FIFO consistency).

# Example

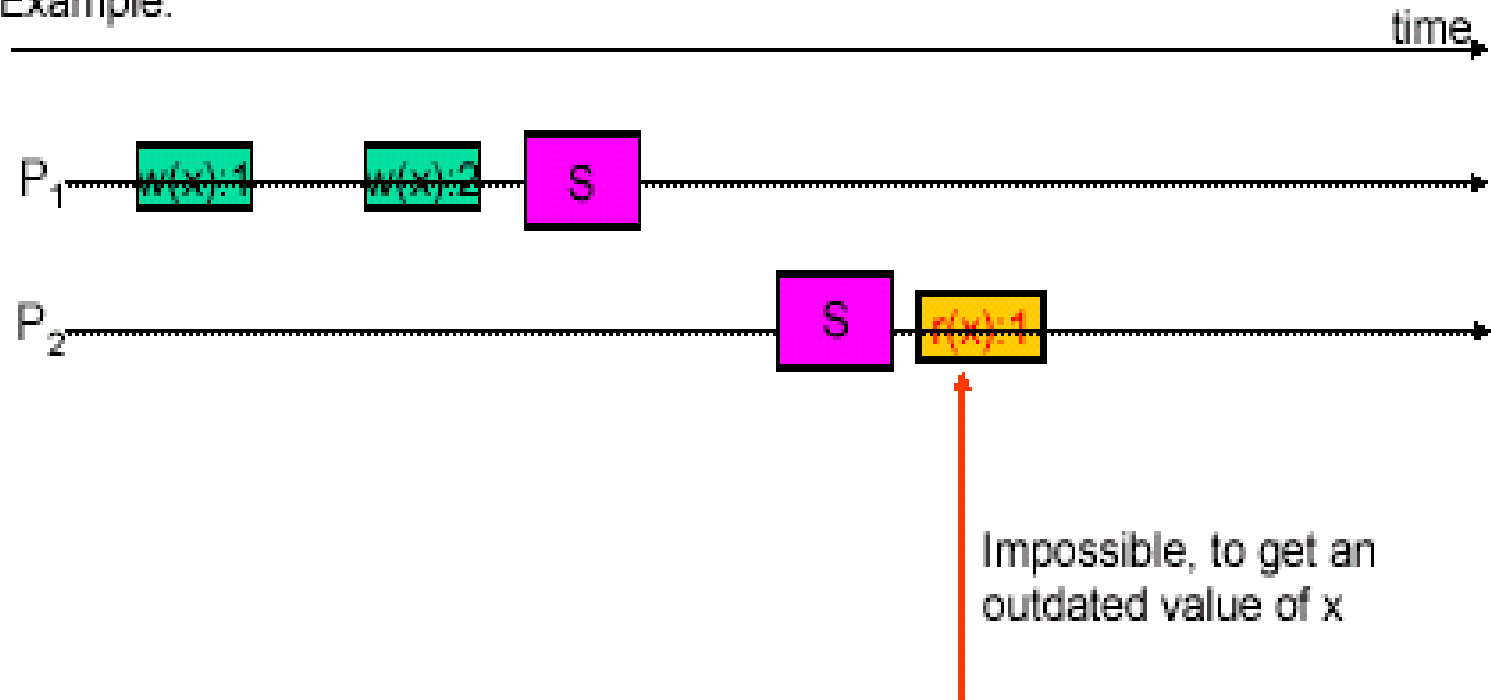
Example:



- Via synchronization you can enforce that you'll get up-to-date values.
- Each process must synchronize if its writes should be seen by others.
- A process requesting a read without any synchronization measures may get out-of-date values.

# Non-weak Consistency

Example:



# Weak Consistency Examples

|           |       |                                                                     |
|-----------|-------|---------------------------------------------------------------------|
| P1: W(x)a | W(x)b | <span style="border: 1px solid red; padding: 0 2px;">S</span>       |
| P2:       | R(x)a | R(x)b <span style="border: 1px solid red; padding: 0 2px;">S</span> |
| P3:       | R(x)b | R(x)a <span style="border: 1px solid red; padding: 0 2px;">S</span> |

(a) before sync., any results are acceptable

|           |       |         |
|-----------|-------|---------|
| P1: W(x)a | W(x)b | S       |
| P2:       |       | S R(x)a |

Wrong!!

- a. A valid sequence of events for weak consistency. This is because P2 and P3 have yet to synchronize, so there's no guarantees about the value in 'x'.
- b. An invalid sequence for weak consistency. P2 has synchronized, so it cannot read 'a' from 'x' – it should be getting 'b'.

# Release Consistency

- **Problems with weak consistency:** When a synch-variable is accessed, the DDS doesn't know whether this is done because a process has finished writing the shared variables or whether it is about reading them.
  - It must take actions required in both cases, namely making sure that all locally initiated writes have been completed (i.e. propagated to all other machines), as well as gathering in all writes from other machines.
- **Provide two operations:** acquire and release
- **Idea:**
  - Distinguish between memory accesses in front of a critical section (acquire) and those behind of a critical section (release).
- **Implementation:**
  - When a release is done, all the protected data that have been updated within the critical section will be propagated to all replicas

# Definition

- Definition: A DDS offers release consistency, if the following three conditions hold:
  1. Before a read or write operation on shared data is performed, all previous acquires done by the process must have completed successfully.
  2. Before a release is allowed to be performed, all previous reads and writes by the process must have been completed
  3. Accesses to synchronization variables are FIFO consistent.

# Example

|     |        |       |        |        |        |
|-----|--------|-------|--------|--------|--------|
| P1: | Acq(L) | W(x)a | W(x)b  | Rel(L) |        |
| P2: |        |       | Acq(L) | R(x)b  | Rel(L) |
| P3: |        |       |        |        | R(x)a  |

- Valid event sequence for release consistency, even though P3 missed to use acquire and release.
- Remark: Acquire is more than a lock or enter\_critical\_section, it waits until all updates on protected data from other nodes are propagated to its local replicas, before it enters the critical section

# Release Consistency

- **Problems with “eager” release consistency:** When a release is done, the process doing the release pushes out **all the modified data to all processes** that already have a copy and thus might potentially read them in the future.
- There is no way to tell if all the target machines will ever use any of these updated values in the future → above solution is a bit inefficient, too much overhead.

# Entry Consistency

- Unlike release consistency, entry consistency requires **each ordinary shared** variable to be protected by a **synchronization variable**.
- When an acquire is done on a synchronization variable, only those ordinary shared variables guarded by that synchronization variable are made consistent.
- A list of shared variables may be assigned to a synchronization variable (to reduce overhead).

|     |         |       |         |       |         |         |
|-----|---------|-------|---------|-------|---------|---------|
| P1: | Acq(Lx) | W(x)a | Acq(Ly) | W(y)b | Rel(Lx) | Rel(Ly) |
| P2: |         |       |         |       | Acq(Lx) | R(x)a   |
| P3: |         |       |         |       |         | R(y)b   |

A valid event sequence for entry consistency

# How to Synchronize?

- Every synch-variable has a current owner
  - An owner may enter and leave critical sections protected by this synchronization variable as often as needed without sending any coordination message to the others.
  - A process wanting to get a synchronization-variable has to send a message to the current owner.
  - The current owner hands over the synch-variable all together with all updated values of its previous writes.
  - Multiple reads in the non-exclusive reads are possible.

# Summary of Consistency Models

## (a) Consistency models not using synchronization operations.

| Consistency     | Description                                                                                                                                |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Strict          | Absolute time ordering of all shared accesses matters.                                                                                     |
| Linearizability | All processes must see all shared accesses in the same order. Accesses are furthermore ordered according to a (nonunique) global timestamp |
| Sequential      | All processes see all shared accesses in the same order. Accesses are not ordered in time                                                  |
| Causal          | All processes see causally-related shared accesses in the same order.                                                                      |
| FIFO            | All processes see writes from each other in the order they were used. Writes from different processes may not always be seen in that order |

## (b) Models with synchronization operations.

| Consistency | Description                                                                                        |
|-------------|----------------------------------------------------------------------------------------------------|
| Weak        | Shared data can be counted on to be consistent only after a synchronization is done                |
| Release     | Shared data are made consistent when a critical region is exited                                   |
| Entry       | Shared data pertaining to a critical region are made consistent when a critical region is entered. |

# Client-Centric Consistency

- Provide guarantees about ordering of operations **only for a single client**, i.e.
  - Effects of an operations **depend** on the client performing it
  - Effects also depend on the history of client's operations
  - Applied only when requested by the client
  - **No guarantees** concerning concurrent accesses by different clients
- Assumption:
  - Clients can access **different replicas**, e.g. mobile users

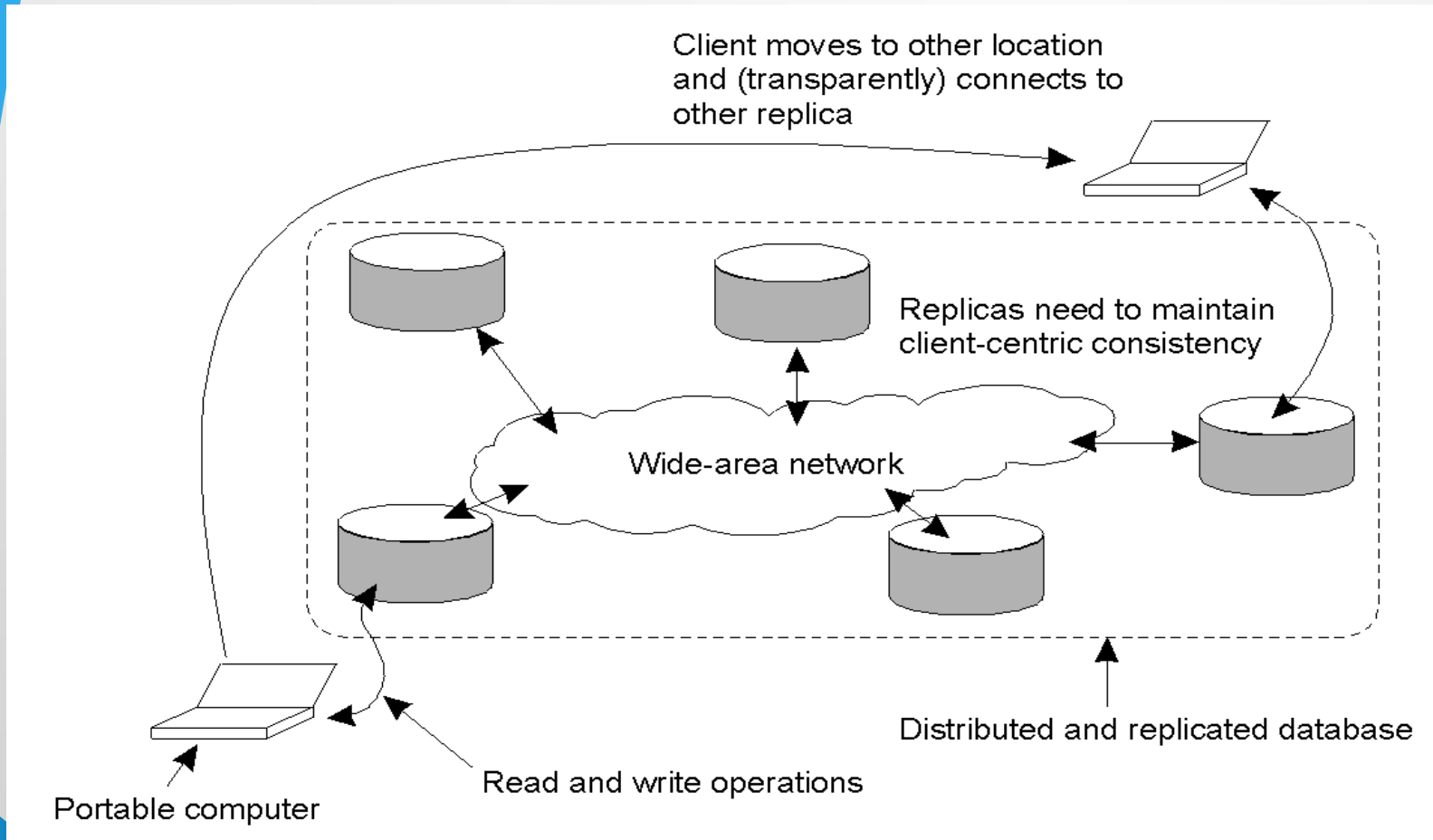
# Eventual Consistency

- If updates do **not** occur for a long period of time, all replicas will gradually become consistent
- Requirements:
  - Few read/write conflicts
  - No write/write conflicts
  - Clients can accept temporary inconsistency
- Examples:
  - DNS:
    - Updates are propagated slowly, and inserts may not be immediately visible.
    - No write/write conflicts
  - WWW:
    - Few write/write conflicts
    - Mirrors eventually updated
  - Cached copies (browser or Proxy) eventually replaced.

# Example: Consistency for Mobile Users

- Consider a distributed database to which you have access through your notebook.
- Assume your notebook acts as a front end to the database.
  - At location A you access the database doing reads and updates.
  - At location B you continue your work, but unless you access the same server as the one at location A, you may detect inconsistencies:
    - your updates at A may not have yet been propagated to B
    - you may be reading newer entries than the ones available at A
    - your updates at B may eventually conflict with those at A
- Note: The only thing you really want is that the entries you updated and/or read at A, are in B the way you left them in A. In that case, the database will appear to be consistent to you.

# Mobile Users



The principle of a mobile user accessing different replicas of a distributed database.

# Client Centric Consistency Models

- Monotonic Reads
- Monotonic Writes
- Read Your Writes
- Writes Follow Reads

# Monotonic Read

- Definition: A DDS provides monotonic-read consistency if the following holds:
  - If process P reads the value of data item x, any successive read operation on x by that process will always return the same value or a more recent one (independently of the replica at location L where this new read will be done).
- Eg-1: Automatically reading your personal calendar updates from different servers. Monotonic reads guarantees that the user sees all updates, no matter from which server the automatic reading takes place.
- Eg-2: Reading (not modifying) incoming mail while you are on the move. Each time you connect to a different e-mail server, that server fetches (at least) all the updates from the server you previously visited

# Example

|       |                  |            |
|-------|------------------|------------|
| L1:   | WS( $x_1$ )      | R( $x_1$ ) |
| <hr/> |                  |            |
| L2:   | WS( $x_1; x_2$ ) | R( $x_2$ ) |

(a)

|       |             |                  |
|-------|-------------|------------------|
| L1:   | WS( $x_1$ ) | R( $x_1$ )       |
| <hr/> |             |                  |
| L2:   | WS( $x_2$ ) | R( $x_2$ )       |
|       |             | WS( $x_1; x_2$ ) |

(b)

The read operations performed by a single process  $P$  at two different local copies of the same data store.

- a. A monotonic-read consistent data store
- b. A data store that does not provide monotonic reads.

# Monotonic Write

- Definition: DDS provides monotonic-write consistency if the following holds:
  - A write operation by process P on data item x is completed before any successive write operation on x by the same process P can take place.
- Remark: Monotonic-writing ~ FIFO consistency
  - Only applies to writes from one client process P
  - Different clients -not requiring monotonic writing may see the writes of process P in any order

# Example

|       |                    |                    |
|-------|--------------------|--------------------|
| L1:   | W(x <sub>1</sub> ) |                    |
| <hr/> |                    |                    |
| L2:   | W(x <sub>1</sub> ) | W(x <sub>2</sub> ) |

(a)

|       |                    |                    |
|-------|--------------------|--------------------|
| L1:   | W(x <sub>1</sub> ) |                    |
| <hr/> |                    |                    |
| L2:   |                    | W(x <sub>2</sub> ) |

(b)

The write operations performed by a single process  $P$  at two different local copies of the same data store

- a) A monotonic-write consistent data store.
- b) A data store that does not provide monotonic-write consistency.

# Examples

- Example-1: Updating a program at server S<sub>2</sub>, and ensuring that all components on which compilation and linking depends, are also placed at S<sub>2</sub>.
- Example-2: Maintaining versions of replicated files in the correct order every where (propagate the previous version to the server where the newest version is installed).

# Read Your Writes

- Definition: DDS provides “read your write” consistency if the following holds:
  - The effect of a write operation by a process  $P$  on a data item  $x$  at a location  $L$  will always be seen by a successive read operation by the same process.
- Example of a missing read-your-write consistency:
  - Updating a website with an editor, if you want to view your updated website, you have to refresh it, otherwise the browser uses the old cached website content.
  - Updating passwords

# Example

L1:  $W(x_1)$

---

L2:  $WS(x_1; x_2)$   $R(x_2)$

(a)

L1:  $W(x_1)$

---

L2:  $WS(x_2)$   $R(x_2)$

(b)

- a) A data store that provides read-your-writes consistency.
- b) A data store that does not.

# Writes Follow Reads

- Definition: DDS provides “writes-follow-reads” consistency if the following holds:
  - A write operation by a process  $P$  on a data item  $x$  following a previous read by the same process, is guaranteed to take place on the same or even a more recent value of  $x$ , than the one having been read before.

(a) A writes-follow-reads consistent data store

|       |                |          |
|-------|----------------|----------|
| L1:   | $WS(x_1)$      | $R(x_1)$ |
| <hr/> |                |          |
| L2:   | $WS(x_1; x_2)$ | $W(x_2)$ |
|       | (a)            |          |

Example: See reactions to posted articles only if you have the original posting (a read “pulls in” the corresponding write operation).

# Summary on Consistency Models

- Choosing the right consistency model requires an analysis of the following trade-offs:
  - Consistency and redundancy
    - All replicas must be consistent
    - All replicas must contain full state
    - Reduced consistency → reduced reliability
  - Consistency and performance
    - Consistency requires extra work
    - Consistency requires extra communication
    - May result in loss of overall performance